

Salt Basin · Build Progress Report

SALT BASIN NET WORKS · BOTTOM LINES WITH A RISING TIDE

THE BUILD AT A GLANCE

Salt Basin Net Works · Build to Date

A snapshot of every product feature shipped on saltbasin.net, generated from the live admin backlog. Includes the platform itself, the multi-tenant CMS for member operators, lead capture, email infrastructure, public-site content, and the supporting deployment + observability layers.

CALENDAR DAYS 35	REQUIREMENTS DELIVERED 69 / 110	TOTAL HOURS 106.3	CLAUDE HOURS 94.1
BETSY HOURS 12.2	ACTIVITIES (C + B) 520 + 55	CLAUDE CODE SPEND \$102.30	AVOIDED TIER COST / YR \$1248.00

Methodology: hours by person are explicit per requirement (editable from the Backlog drawer). Claude spend is calculated at \$0.02/Claude-minute of focused work — calibration documented in code. Tier savings are listed costs of the paid alternatives that were sidestepped.

BUILD PROGRESS OVER TIME

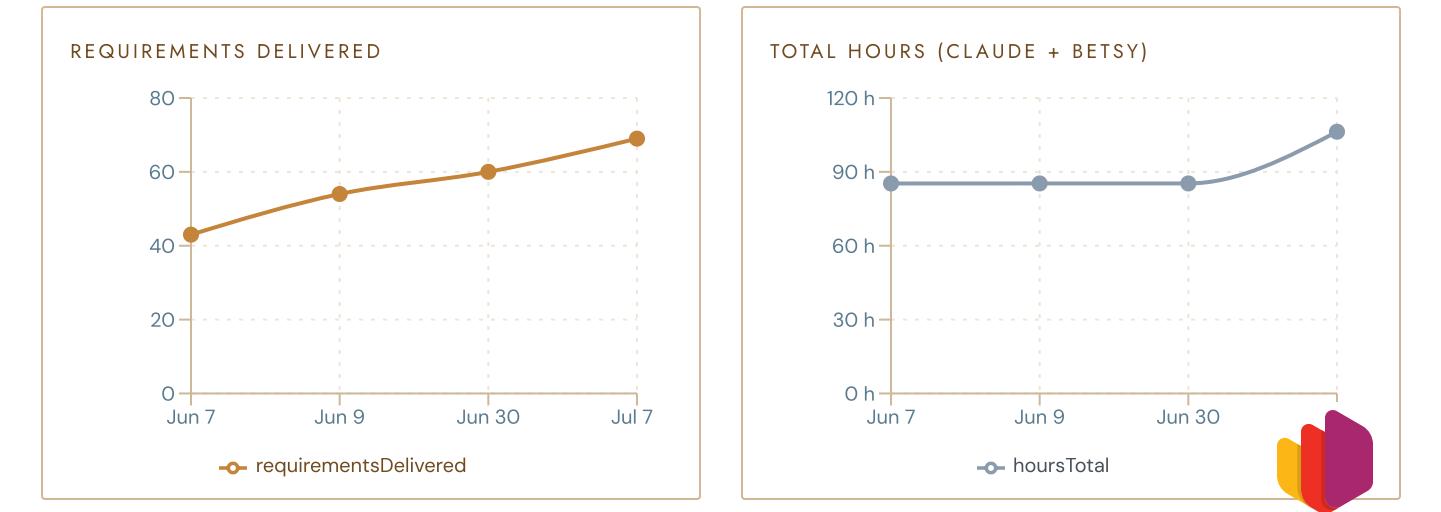
Trajectory across requirements, hours, and AI savings

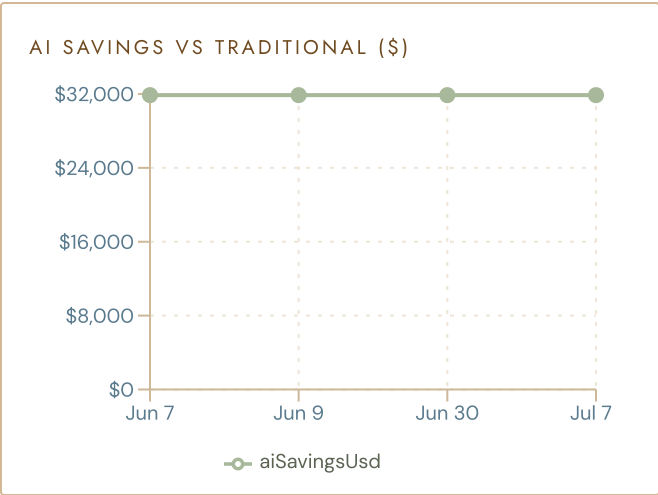
7 DAYS

30 DAYS

90 DAYS

ALL





WHAT THIS WOULD HAVE COST PRE-AI

AI vs traditional dev team comparison

ACTUAL AI-AUGMENTED COST	ESTIMATED TRADITIONAL COST	SAVINGS VS TRADITIONAL	COST MULTIPLIER AVOIDED
\$102.30	\$31980.00	\$31877.70	312.6x

Estimate methodology: a traditional senior-dev team delivering the same scope would need approximately **2.5× the effort hours** (accounting for slower context-switching, more debugging cycles, design review, PM overhead, and code review), at a **\$150/hour blended rate** (US senior full-stack). Applied to the total hours captured per requirement.

DELIVERY TIMELINE

Capabilities in the order they shipped

Platform Foundation 5 requirements · 15.5h (14.6h Claude + 0.9h Betsy) · \$18.60	FIRST SHIPPED JUN 5
Multi-tenant CMS 5 requirements · 6.6h (6.5h Claude + 0.1h Betsy) · \$7.92	FIRST SHIPPED JUN 5
Lead Capture & Identity 4 requirements · 9.5h (9.5h Claude + 0h Betsy) · \$11.40	FIRST SHIPPED JUN 5
Email Infrastructure 4 requirements · 5.5h (4.5h Claude + 1h Betsy) · \$6.60	FIRST SHIPPED JUN 5
Salt Basin Net Works 2 requirements · 3h (3h Claude + 0h Betsy) · \$3.60	FIRST SHIPPED JUN 5



Public Site Content
4 requirements · 15.5h (14.4h Claude + 1.1h Betsy) · \$18.60

FIRST SHIPPED JUN 5

Output Pages (Resume / Proposal / Case Study / One-Pager)
3 requirements · 7.5h (7.5h Claude + 0h Betsy) · \$9.00

FIRST SHIPPED JUN 5

Admin Experience
7 requirements · 11.6h (10.9h Claude + 0.7h Betsy) · \$13.92

FIRST SHIPPED JUN 5

Deployment Infrastructure
3 requirements · 3.5h (3.4h Claude + 0.1h Betsy) · \$4.20

FIRST SHIPPED JUN 5

Security & Data
2 requirements · 2h (2h Claude + 0h Betsy) · \$2.40

FIRST SHIPPED JUN 5

Observability & Quality
1 requirement · 1h (1h Claude + 0h Betsy) · \$1.20

FIRST SHIPPED JUN 5

Requirements & Test Management
3 requirements · 4.1h (3.5h Claude + 0.6h Betsy) · \$4.86

FIRST SHIPPED JUN 5

CAPABILITIES — INVENTORY + TECH STACK + COST

What was built, in what stack, at what cost

Platform Foundation

5 features · h · \$18.60

The shared CMS infrastructure both you and members use — schema, auth, draft/publish, block library.

React 19

Vite

React Router

Express

Postgres (Supabase)

bcryptjs

cookie-parser

CSS variables

► 5 requirements (click to expand)

Multi-tenant CMS

5 features · h · \$7.92

Member admin parity — every operator runs their own multi-page site with the same shell you use, scoped to their own data.

React 19

Express

Postgres (JSONB-style TEXT)

AdminShell shared component

requireUser middleware

► 12 requirements (click to expand)



Lead Capture & Identity

4 features · h · \$11.40

Public forms → leads → match-and-merge dedup → password-protected lead records → optional member conversion.

React 19

Express

Postgres

bcryptjs (lead passwords)

crypto (publicId generation)

lead_sessions cookies

► 8 requirements (click to expand)

Email Infrastructure

4 features · h · \$6.60

Custom-domain mailbox (Zoho), transactional sending (Brevo), DNS auth (SPF / DKIM / DMARC), templated outbound.

Brevo API (v3/smtp/email)

Zoho Mail (Forever Free)

Wix DNS

SPF + CNAME-based DKIM + DMARC

fetch (no SDK)

lead_emails audit table

► 5 requirements (click to expand)

Salt Basin Net Works

2 features · h · \$3.60

The opt-in operator network: members opt in via their config, their card appears in the rotating banner on the Salt Basin home page.

React 19

Express

Postgres

CSS scroll-snap (horizontal banner)

one-shot script (inject-networks-banner.mjs)

► 2 requirements (click to expand)

Public Site Content

4 features · h · \$18.60

Founder-first hero, industries × domains wheel, services, case studies, timeline, technology, contact, references.

React 19

SVG (industry wheel)

CSS Grid + Flexbox

Cormorant Garamond + Inter web fonts

► 5 requirements (click to expand)

Output Pages (Resume / Proposal / Case Study / One-Pager)

3 features · h · \$9.00

Print-friendly, member-gated, deep-link-shareable views of high-value artifacts.

React 19

CSS @media print

GatedPreview component (auth gate)

OutputFrame wrapper

► 6 requirements (click to expand)



Admin Experience

7 features · h · \$13.92

Salt Basin admin UX: sidebar, editor, preview, config panel, leads panel, Net Works roster, mobile.

React 19

multer (image uploads)

Supabase Storage

CSS variables (light/dark adaptive)

mobile breakpoint <768px

► 11 requirements (click to expand)

Deployment Infrastructure

3 features · h · \$4.20

Render (backend), Netlify (frontend), Supabase (DB + storage), GitHub Actions monitors, env-var management.

Render (free tier)

Netlify (free tier)

Supabase (free tier: Postgres + Storage)

GitHub Actions

libsodium-wrappers (encrypted GH secrets)

Render API

Netlify API

► 3 requirements (click to expand)

Security & Data

2 features · h · \$2.40

Auth (bcrypt + sessions), password rotation, data notice, gitignore hygiene, lead-data protection.

bcryptjs

node:crypto (randomBytes)

cookie-based sessions

rotate-admin-password.mjs script

gitignored secret files

► 5 requirements (click to expand)

Observability & Quality

1 feature · h · \$1.20

Deploy monitors, missed-deploy detection, build-credit hygiene, local verify-before-push workflow.

GitHub Actions

UptimeRobot (13-min pings)

Render API polling

auto-issue creation on failure

netlify.toml build.ignore script

► 5 requirements (click to expand)

Requirements & Test Management

3 features · h · \$4.86

This dashboard, deployment correlation, test cases, test runs, defect lifecycle (under construction).

React 19

Express

Postgres (capability_groups + backlog_items + tier_workarounds)

side-drawer pattern

auto-seed on first load

► 13 requirements (click to expand)

TIER WORKAROUNDS — HOW WE STAYED FREE

\$104/mo of avoided product upgrades



Each entry below documents a moment where the default path required a paid tier, and the workaround that kept the build on free products without sacrificing the user experience.

Resend (transactional email)

SAVED \$19.00/MO

Avoided: Resend paid tier + dedicated DNS host

Problem. Resend's domain verification requires an MX record on a subdomain (e.g. send.saltbasin.net) so they can handle bounce notifications. Wix DNS — which hosts saltbasin.net's DNS records — does not allow MX records on subdomains, only at the apex. Apex MX was already taken by Zoho for inbound mail. Resend therefore could not be verified on Wix DNS at any tier.

Workaround. Switched to Brevo (formerly Sendinblue), whose modern verification uses CNAME-based DKIM and a Brevo-domain return-path — no subdomain MX requirement. Brevo's free forever tier (9K emails/month) covers the foreseeable volume, vs Resend's 3K. Code refactor was small (parseFrom helper, payload reshape, header rename).

Zoho Mail (mailbox)

SAVED \$6.00/MO

Avoided: Google Workspace (\$6/user/month) or Microsoft 365 (\$6/user/month)

Problem. A professional consulting brand needs a real custom-domain mailbox (betsy@saltbasin.net) for receiving lead replies, not a Gmail forward. Standard paid options are \$6/user/month minimum, which adds up across an opt-in member network.

Workaround. Used Zoho Mail's Forever Free Plan — custom-domain mailbox, 5GB storage, 1 user, web access only (no IMAP/POP — accept that constraint). Setup is the same standard SPF/DKIM/MX flow as paid providers. Trade-off: members would each need their own paid mailbox eventually, but for the founder identity Zoho free is fully sufficient.

Render (backend hosting)

SAVED \$25.00/MO

Avoided: Render Starter / Pro (\$7-25/mo)

Problem. Render's free tier spins down after 15 minutes of inactivity, causing 20-30 second cold starts on the next visitor. For a consulting brand site, a 20s blank screen is a brand-killer.

Workaround. Two layers: (1) a branded ColdStartLoader component renders immediately with the Salt Basin wordmark and pulse dots so the brand is visible during the cold start, with an explainer message after 3s. (2) UptimeRobot pings /api/health every 13 minutes (under Render's 15-min idle threshold), keeping the container warm during business hours. Combined: cold starts are rare and when they happen, the experience is branded not broken.



Netlify (frontend hosting)

SAVED \$19.00/MO

Avoided: Netlify Starter (\$19/mo) or Pro tier

Problem. Iterative development with many small commits exhausts Netlify's free-tier build minutes (300/mo) quickly. When credits run out, Netlify blocks ALL deploys — both automated CI and manual CLI/drag-drop — until the next billing cycle.

Workaround. Two mitigations: (1) Added a build.ignore script to netlify.toml that exits 0 (skip) when git diff finds no changes in frontend-relevant paths (src/, public/, index.html, vite.config.js, netlify.toml, package*.json) — backend-only commits stop burning credits. (2) Adopted an explicit-approval workflow: Claude asks before pushing any commit that would trigger a Netlify build, so frontend changes get batched. Future option: migrate to Cloudflare Pages (free, unlimited builds) when the next batch of frontend changes is ready.

Supabase (Postgres + Storage)

SAVED \$25.00/MO

Avoided: Supabase Pro (\$25/mo)

Problem. Render's free tier does not include persistent disk for the backend container. Files (image uploads, SQLite databases) would be wiped on every deploy or restart.

Workaround. Migrated all persistent state to Supabase's free tier: Postgres for the relational data (500MB), Storage for image uploads (1GB), with absolute URLs returned by the upload endpoint so the browser fetches images directly from Supabase's CDN (no proxy through Render). Free tier limits are well above current needs.

GitHub Actions (CI / deploy monitoring)

SAVED \$10.00/MO

Avoided: A paid CI provider or self-hosted runner

Problem. Render does not surface deploy failures clearly. A silent skipped auto-deploy is easy to miss until the next deploy unintentionally batches the missed change.

Workaround. Built two GitHub Actions workflows that run on the free tier (2,000 min/mo for public repos): render-deploy-monitor watches every push to main, polls the Render API, and creates a GitHub Issue with the build log on any failure. render-deploy-verify runs ~5 min after push to catch silently-skipped deploys. Cost: \$0; coverage: every deploy.

